

Nikunj Gupta

PS-1 Final Report Nikunj Gupta 2023A7PS0229U.docx

-  Endsem Report
-  PS-1 2025 Endsem Report
-  Birla Institute of Technology & Science Pilani

Document Details

Submission ID

trn:oid:::1:3305724871

Submission Date

Jul 31, 2025, 10:46 AM GMT+4

Download Date

Jul 31, 2025, 12:49 PM GMT+4

File Name

PS-1_Final_Report_Nikunj_Gupta_2023A7PS0229U.docx

File Size

156.3 KB

14 Pages**2,140 Words****13,869 Characters**

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **10 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 3%  Publications
- 8%  Submitted works (Student Papers)

Match Groups

- 10 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 3% Publications
- 8% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	
	BITS, Pilani-Dubai	2%
2	Student papers	
	King's College	2%
3	Student papers	
	University of Glamorgan	1%
4	Internet	
	www.coursehero.com	1%
5	Publication	
	Ammar Aldallal, Faisal Alisa. "Effective Intrusion Detection System to Secure Data..."	<1%
6	Publication	
	K. Silawarawet, N. Nupairoj. "Minimizing Multicast Communication Latency in WA..."	<1%

A REPORT ON
REAL-TIME CYBERSECURITY ATTACK MONITORING AND DETECTION USING AI
AND MACHINE LEARNING

BY
NIKUNJ GUPTA
2023A7PS0229U

Prepared in Fulfillment of the
Practice School – I Course
AT
LEO TECHNOLOGICAL
PUNE, INDIA

A Practice School – I station of
BITS Pilani, Dubai Campus
Dubai International Academic City, Dubai
UAE

(JUNE 2025 – AUG 2025)



Station: LEO TECHNOLOGICAL

Centre: Pune

Duration: JUNE 2025 - AUG 2025

Date of Start: 9-JUNE-2025

Date of Submission: 1-Aug-2025

Title of the Project: Real-Time Cybersecurity Attack Monitoring and Detection Using Machine Learning

ID No. / Name of the students:

NIKUNJ GUPTA(ID:2023A7PS0229U)

DEVANSHI JAIN (Intern from U.W. Maddison university, USA)

Discipline of Students: Computer Science

Name(s) and Designation(s) of the Expert(s):

Mr. Satyen Jain (Director), Mr. Abbas(Team Lead)

Name of the PS Faculty:

Dr. Pranav Mothabhau Pawar

Key Words: Cybersecurity, DDoS Detection, Machine Learning, Kafka, gRPC, Real-Time Monitoring



Abstract

Attacks that target network infrastructure are growing in scale and frequency, forcing security teams to detect malicious traffic within seconds rather than hours. This report documents the design, implementation and evaluation of a cross-platform, real-time detection pipeline that ingests raw packets, extracts 28 statistical flow features and classifies each flow as BENIGN or DDoS ATTACK using an ensemble machine-learning model. The final solution combines high-throughput Linux AF_PACKET capture, lightweight macOS Scapy capture, Kafka-based streaming, multithreaded job servers and a unified REST/gRPC inference service. Extensive experiments demonstrate 2.0 Gbps packet-capture throughput with ~38% CPU and 2.1+ GB memory on Ubuntu 20.04 while maintaining <200 ms end-to-end latency and 99%+ test accuracy on the CIC-DDoS-2019 dataset. The pipeline therefore satisfies enterprise performance and accuracy requirements.

Acknowledgements

I would like to give my acknowledgement and thanks to the Director, BPDC, and the Dean, Practice School division, with whose support I have been able to pursue the PS I programme. I would like to give special thanks to the Head of the PS Station Mr. Satyen Jain, the PS Coordinator Prof. Pranav, industry mentor Mr. Abbas and the network operations centre whose traffic samples enabled realistic stress testing. Gratitude is also extended to the developers of Apache Kafka and the maintainers of CIC-DDoS-2019 for providing indispensable open-source tools and datasets.

Contents

- Introduction
- Literature Background
- System Architecture
- Flow State Management
- Streaming Pipeline
- Machine-Learning Component
- API & Deployment
- Performance Evaluation
- Challenges & Solutions
- Future Work
- Conclusion
- List of Tables
- Appendix A: Feature Definitions
- Appendix B: Acronyms

List of Tables

Table No.	Title	Page
1	Multi-Platform Packet Capture Performance	6
2	Ensemble Model Configuration	8
3	End-to-End Latency Breakdown	9
4	Challenges and Solutions Summary	10

Introduction

Motivation

Distributed-Denial-of-Service attacks can saturate links within seconds, overwhelming security information and event management (SIEM) consoles that rely on batch log analysis. A proactive defense requires continuous packet capture, rapid feature extraction and sub-second classification.

Objectives

1. Capture packets on Linux and macOS at line rate without vendor-specific hardware.
2. Maintain per-flow statistics for up to 300 s while preventing memory exhaustion through timeouts.
3. Stream enriched flow records to horizontally scalable job servers via Apache Kafka.
4. Classify each flow using an ensemble ML model trained on 28 features aligned with the CIC-DDoS-2019 schema.
5. Expose predictions over both REST and gRPC to allow heterogeneous security toolchains to consume alerts.

Literature Background

Sharafaldin et al. created a realistic multi-vector DDoS dataset that remains the de-facto benchmark for academic work in the field. Kreps et al. introduced Apache Kafka, demonstrating its suitability for high-volume log processing. Recent studies advocate combining statistical flow features with deep-learning modules such as CNNs to improve generalization to zero-day traffic, a direction considered for future enhancement in this project.

System Architecture

Multi-Platform Packet Capture Layer

Table 1: Multi-Platform Packet Capture Performance

Platform	Capture Method	Throughput	CPU Usage	Memory
Ubuntu 20.04	AF_PACKET	2.0 Gbps	38%	2.1 GB
macOS 12.0	Scapy	1.2 Gbps	52%	2.3 GB
CentOS 8	AF_PACKET	1.9 Gbps	41%	2.0 GB

Linux AF_PACKET Backend

Raw-socket capture avoids context switches inherent in pcap libraries, doubling throughput compared with libpcap tests on identical hardware. The implementation utilizes socket-level packet filtering to minimize user-space transitions while maintaining compatibility with standard network interfaces.

macOS Scapy Backend

Native BSD packet filters provide safe user-space access where raw sockets are restricted. Although Python-level overhead increases CPU utilization to 52% at 1.2 Gbps, this remains acceptable for Apple M-series laptops used in field trials.

Flow State Management

The system maintains per-flow statistics using efficient hash-based lookups keyed by the bidirectional five-tuple. A dedicated FlowStats container tracks packet counts, byte totals, inter-arrival times and selected flag frequencies without buffering full payloads, thereby satisfying GDPR data-minimization guidelines.

Flow timeout mechanisms prevent memory exhaustion during high-traffic periods, automatically evicting inactive flows after 300 seconds. This approach balances memory efficiency with the need to capture long-lived sessions such as video streams or large file transfers.

The hash-table implementation uses Python's defaultdict structure with UUID-based flow identification, ensuring collision-free tracking of concurrent sessions. Statistical calculations are performed incrementally to minimize computational overhead during packet processing.

Streaming Pipeline

Kafka Topic Configuration

- **Topic Name:** `network_flows`
- **Producer Configuration:** The capture modules serialize each flow into JSON and write to Kafka with the source IP as the key, ensuring partition affinity for hotspot analysis.
- **Consumer Group:** Job servers fetch batches using group ID `job_server_group`, enrich metadata and forward to the ML micro-service. Auto-offset reset is configured to latest to minimize replay lag, while the group configuration guarantees at-least-once delivery.

Multithreaded Job Servers

A thread-pool sized at $2 \times \text{CPU}$ cores processes messages concurrently, amortizing I/O latency when calling the ML service. The implementation uses ThreadPoolExecutor with a bounded queue of 10,000 elements; excess flows are dropped with explicit log entries, preventing memory blow-ups during volumetric attacks.

Back-pressure management ensures system stability during peak traffic periods. The processor threads continuously poll the Kafka consumer, extract flow features, enrich with metadata (request IDs, timestamps), and forward to the ML prediction service via REST API calls.

Machine-Learning Component

Dataset & Pre-processing

The CIC-DDoS-2019 CSV includes 88 raw attributes per packet flow. Pre-processing maps original column headers to the exact 28-feature API contract through a comprehensive mapping dictionary. Missing columns observed during real-world capture are zero-filled to avoid runtime crashes.

Class Balancing

Random under-sampling equalizes Attack and Benign samples at 100,000:300,000, preventing bias towards the majority benign class. This approach ensures balanced training while maintaining computational efficiency during model training phases.

Feature Scaling

StandardScaler normalizes variance to accelerate convergence of gradient-boosting models. The scaler is fitted during training and preserved with the model for consistent preprocessing during inference.

Ensemble Configuration

Table 2: Ensemble Model Configuration

Sub-Model	Hyper-Parameters	Purpose
Random Forest	1,000 trees, depth 20, class weight balanced	Baseline variance reduction
XGBoost	1,000 estimators, LR 0.05, colsample 0.9	Captures complex non-linear patterns
LightGBM	1,000 estimators, depth 10, learning_rate 0.05	Efficient on large tabular data

A soft-voting classifier aggregates probability outputs, achieving 99.2% test accuracy and 0.002 standard deviation across five folds using StratifiedKFold cross-validation.

Binary Decision Threshold

The production server flags a flow as malicious only when the predicted attack probability exceeds 90%, reducing false positives fed into SOC ticket queues. This threshold was empirically determined through ROC curve analysis to optimize the precision-recall trade-off.

API & Deployment

Unified FastAPI + gRPC Server

- **REST Endpoint:** `/predict` accepts a single JSON flow and returns `PredictionResponse` with latency metrics and confidence scores.
- **gRPC Service:** `DDoSDetectionService` exposes `Predict`, `PredictBatch` and `HealthCheck` RPCs for low-overhead, high-frequency integrations with Nginx WAF plugins.

- **Model Loading:** The server auto-loads the trained model at start-up, logging the object type, feature order and version string to facilitate A/B deployments.

Health & Observability

The /health endpoint returns model status, uptime and feature-count so that Kubernetes liveness probes can act upon degraded states. Structured JSON logs include request IDs propagated end-to-end via Kafka for distributed tracing capabilities.

Cross-platform compatibility is ensured through Docker containerization, with separate base images optimized for Linux and macOS deployment scenarios.

Performance Evaluation

End-to-End Latency

Table 3: End-to-End Latency Breakdown

Stage	Mean Latency	95th Percentile	Notes
Packet→Kafka	9 ms	14 ms	Linux AF_PACKET on 1 Gbps link
Kafka→JobServer dequeue	4 ms	6 ms	Queue length <70% capacity
JobServer→ML REST	38 ms	60 ms	Includes JSON serialization
ML Inference	22 ms	40 ms	Ensemble predict_proba() CPU only
Full Pipeline	73 ms	118 ms	Meets sub-200 ms SLA

Throughput Stress Test

Running four concurrent processing threads saturates 1.0 Gbps input without packet loss on commodity hardware (Intel i3, 10 GB RAM). Garbage collection cycles remain bounded; the processing queue drains within 250 ms even during SYN-flood bursts.

Memory utilization stabilizes at ~2.1 GB during sustained traffic, with flow table pruning maintaining consistent performance across extended test periods.

Challenges & Solutions

Table 4: Challenges and Solutions Summary

Challenge	Impact	Solution Implemented
High false positives in imbalanced dataset	Alert fatigue	Random under-sampling + 90% threshold
Cross-platform capture driver incompatibility	Deployment delays	Abstracted capture into backend strategy pattern
Large flow feature gaps during live capture	Runtime exceptions	Zero-fill & NaN-to-zero policy in feature extraction
Limited SIEM integration	Manual analyst triage	JSON output plus roadmap for enterprise connectors

Multi-Platform Architecture

Cross-platform packet capture implementation supporting both Linux AF_PACKET and macOS Scapy backends ensures broad deployment compatibility. The strategy pattern abstraction allows seamless switching between capture methods based on the underlying operating system.

Reduced False Positive Rates

Strategic class balancing through random under-sampling combined with optimized decision thresholds significantly reduced false positive rates, improving operational efficiency and reducing alert fatigue for security operations teams.

Future Work

1. **CNN-based Feature Learning** – Replace handcrafted statistics with embeddings learned directly from packet-header sequences, expected to reduce unnoticed volumetric attack variants.
2. **Adaptive Thresholding** – Dynamic adjustment based on baseline network conditions to improve precision during peak hours and varying traffic patterns.
3. **Native SIEM Connectors** – Out-of-the-box Splunk HEC and Elastic Beats modules to lower integration effort for enterprise security platforms.
4. **Edge Deployment** – Compile lightweight inference models to WebAssembly for runtime inside Nginx modules, cutting hop latency below 10 ms for real-time protection.

Future developments could incorporate deep learning approaches for automatic feature discovery. Convolutional Neural Networks (CNNs) applied to packet header sequences may reveal attack patterns not captured by traditional statistical features.

Development of native connectors for popular SIEM platforms (Splunk, ELK Stack, IBM QRadar) would facilitate enterprise deployment. Standardized alert formats and workflow integration remain important considerations for operational acceptance.

Conclusion

The project delivers a fully operational, cross-platform DDoS detection pipeline that combines high-speed packet capture, Kafka-based streaming and a robust ensemble ML back-end. Benchmarks confirm 2.0 Gbps capture, <120 ms pipeline latency and 99% detection accuracy on public datasets—all without disclosing proprietary implementation details.

The architecture's modular design and dual REST/gRPC interface lay a foundation for future deep-learning enhancements and SIEM integrations essential for modern enterprise SOC environments. The solution demonstrates that high-performance network security monitoring can be achieved using open-source tools and commodity hardware.

Key achievements include successful cross-platform deployment, sub-second response times, and enterprise-grade accuracy metrics that meet or exceed industry benchmarks for DDoS detection systems.

Appendix A: Feature Definitions

#	Feature Name	Description
01	protocol	Transport-layer protocol number extracted from IP header
02	flow_duration	Time between first and last packet of the flow (μ s)
03	total_fwd_packets	Count of packets from initiator to responder
04	total_backward_packets	Count of packets from responder to initiator
05	fwd_packet_length_max	Maximum payload length of forward packets (bytes)
06	fwd_packet_length_min	Minimum payload length of forward packets (bytes)
07	fwd_packet_length_mean	Mean payload length of forward packets (bytes)
08	packet_length_mean	Mean payload length across both directions
09	packet_length_std	Standard deviation of payload lengths
10	flow_bytes_per_second	Aggregate byte rate over flow lifetime
11	flow_packets_per_second	Aggregate packet rate over flow lifetime
12	flow_iat_mean	Mean inter-arrival time for all packets (μ s)
13	flow_iat_std	Std dev of inter-arrival times (μ s)
14	flow_iat_max	Maximum inter-arrival time (μ s)
15	flow_iat_min	Minimum inter-arrival time (μ s)
16	fwd_iat_total	Sum of forward IATs (μ s)
17	fwd_iat_mean	Mean forward IAT (μ s)
18	fwd_iat_std	Std dev forward IAT (μ s)
19	fwd_iat_max	Maximum forward IAT (μ s)
20	fwd_iat_min	Minimum forward IAT (μ s)
21	bwd_iat_total	Sum of backward IATs (μ s)
22	bwd_iat_mean	Mean backward IAT (μ s)
23	bwd_iat_std	Std dev backward IAT (μ s)
24	bwd_iat_max	Maximum backward IAT (μ s)

25	bwd_iat_min	Minimum backward IAT (μ s)
26	fwd_psh_flags	Forward PSH flag occurrences
27	bwd_psh_flags	Backward PSH flag occurrences
28	fwd_urg_flags	Forward URG flag occurrences

Appendix B: Acronyms

Term	Expansion
AF_PACKET	Linux Address-Family Packet Interface
CNN	Convolutional Neural Network
DDoS	Distributed Denial-of-Service
IAT	Inter-Arrival Time
ML	Machine Learning
PSH	Push Flag (TCP)
SIEM	Security Information & Event Management
SOC	Security Operations Centre
SLA	Service Level Agreement
WAF	Web Application Firewall

References:

Kreps, J., Narkhede, N., Rao, J., et al. (2011). Kafka: a distributed messaging system for log processing. In Proceedings of the NetDB (Vol. 11, pp. 1-7).

"Sharafaldin, I., Lashkari, A. H., Hakak, S., & Ghorbani, A. A. (2019)." ("Sci-Hub | Developing Realistic Distributed Denial of Service (DDoS ...)") Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. In 2019 International Carnahan Conference on Security Technology (ICCST) (pp. 1-8). IEEE.